



WELCOME





C++ Utility Functions

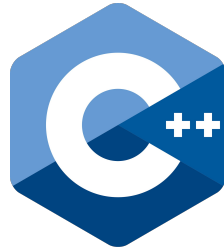
Agenda

Sorting

Next
Permutation

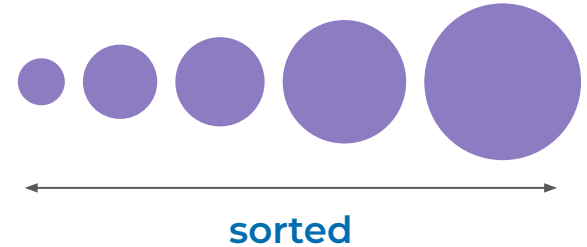
Max/Min/Find
Element

Sort Function



Sorting in C++ STL

- ✓ **Sorting** is the process of arranging elements in a particular order (**ascending/descending**).
- ✓ **C++** provides an efficient in-built sorting function: **sort()**.
- ✓ Sorting improves **searching efficiency** in applications.



Basic Syntax and library

Syntax 

```
#include <algorithm>
```

-
-
-

```
sort(start, end);
```

C++ provides the `sort()` function from the `<algorithm>` library to efficiently sort elements in a container such as `arrays`, `vectors`, and other iterable sequences.

Basic Syntax and library

Syntax 

```
#include <algorithm>
```

-
-
-

```
sort(start, end);
```

start: Iterator pointing to the **first element** of the range to be sorted.

Basic Syntax and library

Syntax 

```
#include <algorithm>
```

-
-
-

```
sort(start, end);
```

end: Iterator pointing **one past the last element** of the range to be sorted.

Basic Syntax and library

Syntax 

```
#include <algorithm>
```

-
-
-

```
sort(start, end);
```

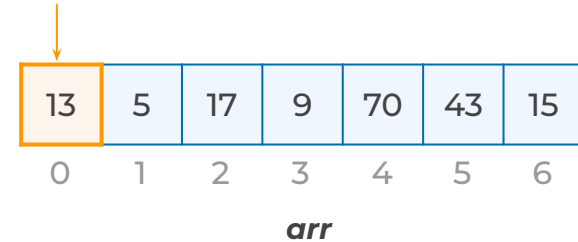
The sorting includes **start** but
excludes end

Sorting

Sorting an Array

C++ 

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    int arr[] = {13, 5, 17, 9, 70, 43, 15};
    int n = 7;
    sort(arr, arr + n);
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

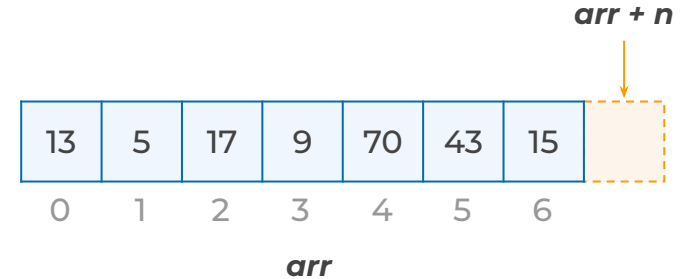


Sorting

Sorting an Array

C++ 

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    int arr[] = {13, 5, 17, 9, 70, 43, 15};
    int n = 7;
    sort(arr, arr + n);
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```



Sorting an Array

C++ 

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    int arr[] = {13, 5, 17, 9, 70, 43, 15};
    int n = 7;
    sort(arr, arr + n);
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

Sorts the array in ascending order.

Output

5 9 13 15 17 43 70

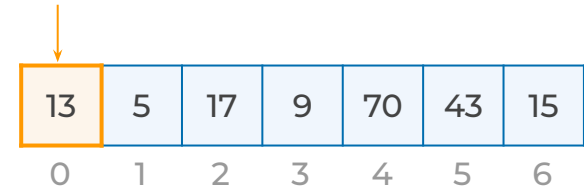
Sorting

Sorting a Vector

C++ 

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    vector<int> v = {13, 5, 17, 9, 70, 43, 15};
    sort(v.begin(), v.end());
    for (int num : v) {
        cout << num << " ";
    }
    return 0;
}
```

begin()



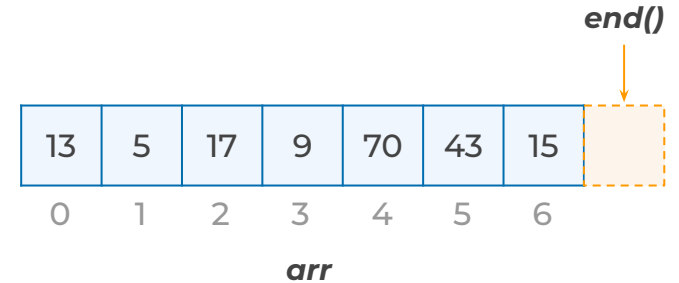
arr

Sorting

Sorting a Vector

C++ 

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    vector<int> v = {13, 5, 17, 9, 70, 43, 15};
    sort(v.begin(), v.end());
    for (int num : v) {
        cout << num << " ";
    }
    return 0;
}
```



Sorting a Vector

C++ 

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    vector<int> v = {13, 5, 17, 9, 70, 43, 15};
    sort(v.begin(), v.end());
    for (int num : v) {
        cout << num << " ";
    }
    return 0;
}
```

Output

5 9 13 15 17 43 70

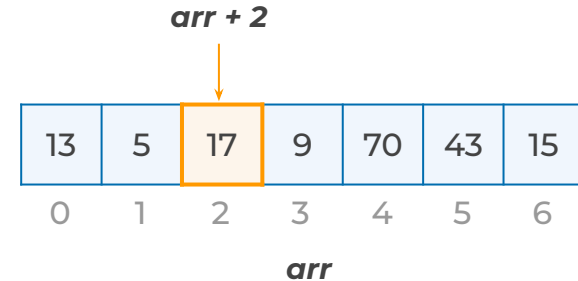
Sorts the vector in ascending order.

Sorting a Subarray

C++ 

```
int arr[] = {13, 5, 17, 9, 70, 43, 15};  
sort(arr + 2, arr + 5);
```

Sorts elements from index 2 to 4.

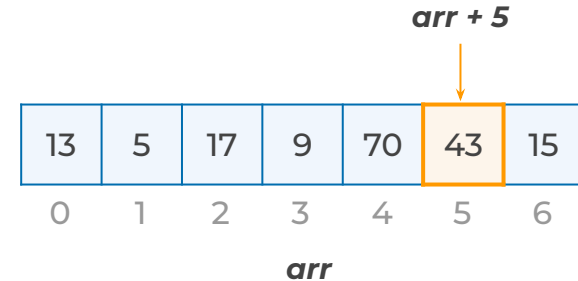


Sorting a Subarray

C++ 

```
int arr[] = {13, 5, 17, 9, 70, 43, 15};  
sort(arr + 2, arr + 5);
```

Sorts elements from index 2 to 4.

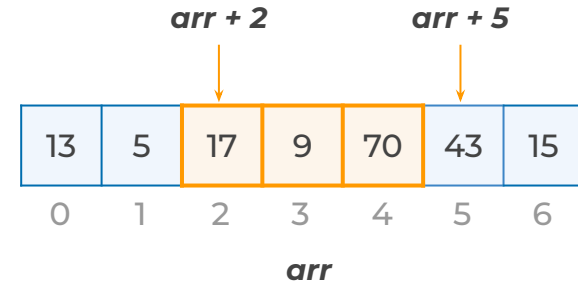


Sorting a Subarray

C++ 

```
int arr[] = {13, 5, 17, 9, 70, 43, 15};  
sort(arr + 2, arr + 5);
```

Sorts elements from index 2 to 4.



Sorting a Subarray

C++ 

```
int arr[] = {13, 5, 17, 9, 70, 43, 15};  
sort(arr + 2, arr + 5);
```

Sorted elements from index 2 to 4.

After Sorting ($arr + 2$ to $arr + 5$)

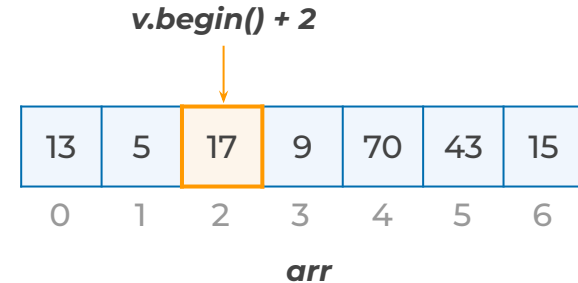
13	5	9	17	70	43	15
0	1	2	3	4	5	6
<i>arr</i>						

Sorting a range in a Vector

C++ 

```
vector<int> v = {13, 5, 17, 9, 70, 43, 15};  
sort(v.begin() + 2, v.begin() + 5);
```

Sorts elements from index 2 to 4.



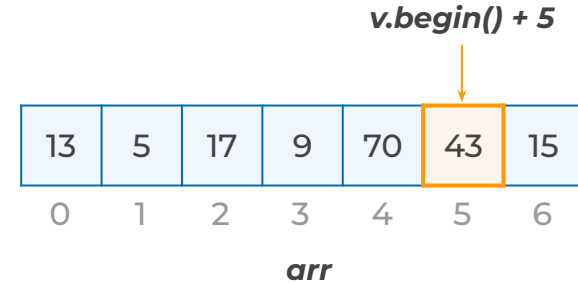
Sorting

Sorting a range in a Vector

C++ 

```
vector<int> v = {13, 5, 17, 9, 70, 43, 15};  
sort(v.begin() + 2, v.begin() + 5);
```

Sorts elements from index 2 to 4.

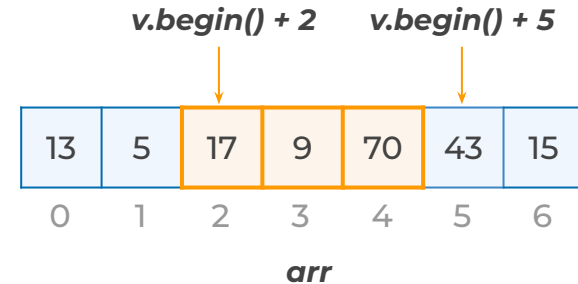


Sorting a Subarray

C++ 

```
int arr[] = {13, 5, 17, 9, 70, 43, 15};  
sort(arr + 2, arr + 5);
```

Sorts elements from index 2 to 4.



Sorting a range in a Vector

C++ 

```
vector<int> v = {13, 5, 17, 9, 70, 43, 15};  
sort(v.begin() + 2, v.begin() + 5);
```

Sorted elements from index 2 to 4.

After Sorting ($v.begin() + 2$ to $v.begin() + 5$)

13	5	9	17	70	43	15
0	1	2	3	4	5	6

arr

Sorting Array in Descending Order

```
int arr[] = {13, 5, 17, 9, 70, 43, 15};  
sort(arr, arr + n, greater<int>());
```

The `greater<int>()` function is a **comparator** used to **sort elements** in **descending order** by **comparing** them in **reverse**.

Sorting Array in Descending Order

C++ 

```
int arr[] = {13, 5, 17, 9, 70, 43, 15};  
sort(arr, arr + n, greater<int>());
```

arr is sorted in descending order

Output

70 43 17 15 13 9 5

Descending Order

Sorting Vector in Descending Order

C++ 

```
vector<int> v = {13, 5, 17, 9, 70, 43, 15};  
sort(v.begin(), v.end(), greater<int>());
```

Output

70 43 17 15 13 9 5

Descending Order

Works similarly to arrays but with vector.

Custom Comparator

C++ 

```
pair<int, int> arr[] = {{8, 4}, {5, 2}, {8, 6}};  
int n = 3;  
sort(arr, arr + n, customComparator);
```

customComparator

- It's a boolean function used to define custom sorting logic.
- It should return **true** if the first element should come before the second, and **false** otherwise.

Custom Comparator

C++ 

```
pair<int, int> arr[] = {{8, 4}, {5, 2}, {8, 6}};  
int n = 3;  
sort(arr, arr + n, customComparator);
```

sort(arr, arr + n, customComparator)

- This sorts the array using the **custom comparator**.
- The elements are compared according to the **logic** defined in the comparator.

Custom Sorting (Using Comparator)

Example

C++ 

```
bool customComparator(pair<int, int> p1, pair<int, int> p2) {  
    if (p1.first < p2.first) return true;  
    if (p1.first > p2.first) return false;  
    return p1.second > p2.second;  
}
```

After the comparison, the 'sort' function will arrange the pairs first by the first element in **ascending order** and then by the second element in **descending order**.

Custom Sorting (Using Comparator)

Example

C++ 

```
#include <iostream>
#include <algorithm>
using namespace std;
bool customComparator(pair<int, int> p1, pair<int, int> p2) {
    if (p1.first < p2.first) return true;
    if (p1.first > p2.first) return false;
    return p1.second > p2.second;
}
int main() {
    pair<int, int> arr[] = {{8, 4}, {5, 2}, {8, 6}};
    int n = 3;
    sort(arr, arr + n, customComparator);
    for (auto p : arr) {
        cout << "(" << p.first << ", " << p.second << ") ";
    }
    return 0;
}
```

Before Sorting

(8,4) (5,2) (8,6)

Custom Sorting (Using Comparator)

Example

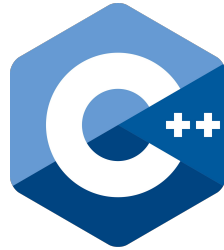
C++ 

```
#include <iostream>
#include <algorithm>
using namespace std;
bool customComparator(pair<int, int> p1, pair<int, int> p2) {
    if (p1.first < p2.first) return true;
    if (p1.first > p2.first) return false;
    return p1.second > p2.second;
}
int main() {
    pair<int, int> arr[] = {{8, 4}, {5, 2}, {8, 6}};
    int n = 3;
    sort(arr, arr + n, customComparator);
    for (auto p : arr) {
        cout << "(" << p.first << ", " << p.second << ") ";
    }
    return 0;
}
```

After Sorting

(5,2) (8,6) (8,4)

Sorting a 2D Vector



Sorting a 2D Vector by Rows (Default Sorting)

Example

C++ 

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    vector<vector<int>> vec = {{3, 2, 5}, {1, 7, 6}, {4, 9, 8}};
    sort(vec.begin(), vec.end());
    for (auto row : vec) {
        for (int num : row) {
            cout << num << " ";
        }
        cout << endl;
    }
    return 0;
}
```

Sorting a **2D vector** by rows sorts each row individually in ascending order based on the first element of each row. If the first elements are the same, it sorts **lexicographically** using the subsequent elements

Sorting a 2D Vector by Rows (Default Sorting)

Example

C++ 

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    vector<vector<int>> vec = {{3, 2, 5}, {1, 7, 6}, {4, 9, 8}};
    sort(vec.begin(), vec.end());
    for (auto row : vec) {
        for (int num : row) {
            cout << num << " ";
        }
        cout << endl;
    }
    return 0;
}
```

After Sorting

1 7 6

3 2 5

4 9 8

Sorting a 2D Vector in Descending Order

Example

C++ 

```
vector<vector<int>> vec = {{3, 2, 5}, {1, 7, 6}, {4, 9, 8}};  
sort(vec.begin(), vec.end(), greater<vector<int>>());
```

Before Sorting

3 2 5

1 7 6

4 9 8

Sorting a 2D Vector in Descending Order

Example

C++ 

```
vector<vector<int>> vec = {{3, 2, 5}, {1, 7, 6}, {4, 9, 8}};  
sort(vec.begin(), vec.end(), greater<vector<int>>());
```

After Sorting

4 9 8

3 2 5

1 7 6

*Sorts each row in descending order by the first element,
and lexicographically if the elements are the same.*

Sorting a 2D Vector by a Specific Column

Example

C++ 

```
bool columnComparator(vector<int> &a, vector<int> &b)
{
    return a[1] < b[1];
}

sort(vec.begin(), vec.end(), columnComparator);
```

Sort based on the second column.

Before Sorting by Column 1

3 2 5

1 7 6

4 9 8

Sorting a 2D Vector by a Specific Column

Example

C++ 

```
bool columnComparator(vector<int> &a, vector<int> &b)
{
    return a[1] < b[1];
}

sort(vec.begin(), vec.end(), columnComparator);
```

Sort based on the second column.

After Sorting by Column 1

3 2 5

1 7 6

4 9 8

Sorting a 2D Vector in Custom Order (Multiple Conditions)

Example

C++ 

```
bool customComparator(vector<int> &a, vector<int> &b)
{
    if (a[0] != b[0]) return a[0] < b[0];
    return a[1] > b[1];
}

sort(vec.begin(), vec.end(), customComparator);
```

Sort by first column (ascending).

Before Sorting

3 2 5
1 7 6
4 9 8
3 5 1

Sorting a 2D Vector in Custom Order (Multiple Conditions)

Example

C++ 

```
bool customComparator(vector<int> &a, vector<int> &b)
{
    if (a[0] != b[0]) return a[0] < b[0];
    return a[1] > b[1];
}

sort(vec.begin(), vec.end(), customComparator);
```

After Sorting

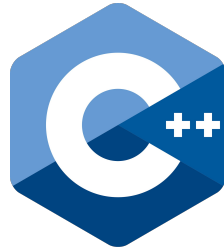
1 7 6
3 5 1
3 2 5
4 9 8

*If first column is equal, sort by second column
(descending).*

Quiz Time!



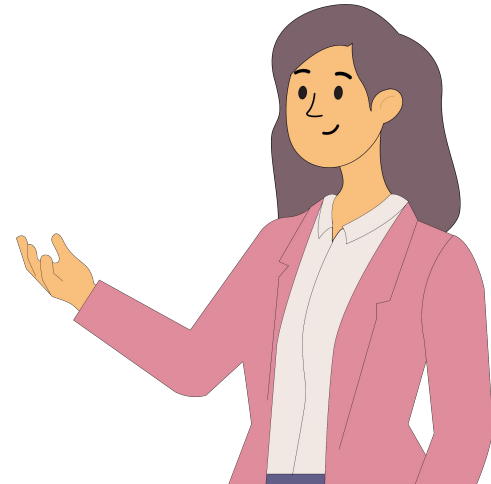
Max element, min element,
find element



Max element, min element, find element

Finding and Searching

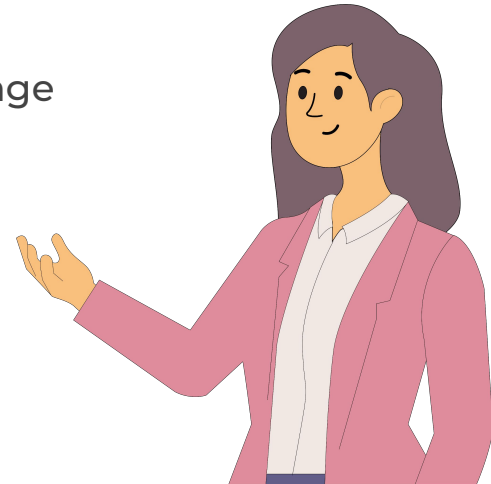
- ✓ The **C++ STL** provides built-in functions for performing common operations such as finding the **maximum**, **minimum**, and **searching** for elements in various containers, including **arrays**, **vectors**, and other iterable sequences.



About

What does `max_element(first,last)` do?

- ✓ The `max_element()` function is part of the `<algorithm>` header in `C++`. It finds the largest element in a given range and returns an iterator to that element.



Finding the Maximum Element

About

Input



- A range defined by two iterators: (**first**, **last**)
- **first**: Iterator pointing to the beginning of the range.
- **last**: Iterator pointing one past the last element of the range.

Output



- An iterator pointing to the **maximum element** in the range.
- If the range is **empty**, **max_element** (**first**,**last**) returns iterator last.

Important note



*The result of **max_element**() is an iterator, not the value itself.*

*To get the actual value, you need to dereference the iterator using * (e.g., ***max_element**(arr, arr + n)).*

Finding the Maximum Element (in an array)

Example

C++

```
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    int arr[] = {13, 5, 17, 9, 70, 43, 15};
    int n = 7;
    int maxVal = *max_element(arr, arr + n);
    cout << "Max Element: " << maxVal << endl;
    return 0;
}
```

Output

Max Element: 70

Finding the Maximum Element (in a Vector)

Example

C++

```
vector<int> v = {13, 5, 17, 9, 70, 43, 15};  
int maxVal = *max_element(v.begin(),  
v.end());
```

Output

Max Element: 70

*`max_element(v.begin(), v.end())` returns an iterator to the maximum element in the vector.
The `*` operator dereferences the iterator to get the actual, maximum value.*

About

What does `min_element(first,last)` do?

- ✓ The `min_element()` function is part of the `<algorithm>` header in `C++`.
- ✓ It finds the **smallest element** in a given range and returns an iterator to that element.



Finding the Minimum Element

About

Input



- A range defined by two iterators: (**first**, **last**)
- **first**: Iterator pointing to the beginning of the range.
- **last**: Iterator pointing one past the last element of the range.

Output



- An iterator pointing to the **minimum element** in the range.
- If the range is **empty**, **min_element** (**first**,**last**) returns iterator **last**.

Important note



*The result of **min_element**() is an iterator, not the value itself.*

*To get the actual value, you need to dereference the iterator using ***** (e.g., ***min_element(arr, arr + n)**).*

Finding the Minimum Element (in an array)

Example

C++

```
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    int arr[] = {13, 5, 17, 9, 70, 43, 15};
    int n = 7;
    int minVal = *min_element(arr, arr + n);
    cout << "Min Element: " << minVal << endl;
    return 0;
}
```

Output

Min Element: 5

Finding the Minimum Element(in a Vector)

Example

C++

```
vector<int> v = {13, 5, 17, 9, 70, 43, 15};  
int minVal = *min_element(v.begin(), v.end());
```

Output

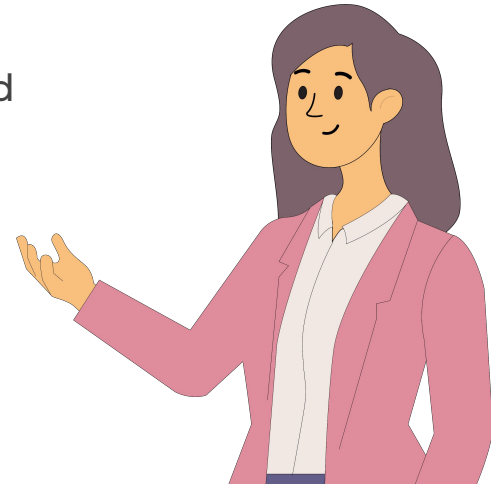
Min Element: 5

*`min_element(v.begin(), v.end())` returns an iterator to the minimum element in the vector.
The `*` operator dereferences the iterator to get the actual, minimum value.*

About

What does `find()` do?

- ✓ The `find()` function is part of the `<algorithm>` header in `C++`. It searches for a given target element in a specified range and returns an iterator to its first occurrence.



Finding an Element

About

Input



- A range defined by two iterators and a target element: (**first**, **last**, **target**)
- **first**: Iterator pointing to the beginning of the range.
- **last**: Iterator pointing one past the last element of the range.
- **target**: The element to search for in the given range.

Output



- An iterator pointing to the **first occurrence** of target in the range.
- If **target** is not found, **find()** returns **last**.

Important note



*The result of **find()** is an iterator, not the value itself.*

*To get the actual value, you need to dereference the iterator using * (e.g., ***find(first, last, target)**).*

Finding an Element

Example

C++

```
int arr[] = {13, 5, 17, 9, 70, 43, 15};  
auto it = find(arr, arr + n, 17);  
if (it != arr + n) {  
    cout << "Element found at index: " << (it - arr);  
} else {  
    cout << "Element not found";  
}
```

Output

Element found at index: 2

Searching in a Vector

Example

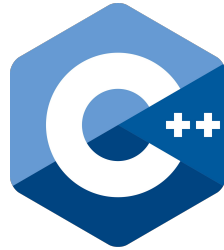
C++

```
vector<int> v = {13, 5, 17, 9, 70, 43, 15};  
auto it = find(v.begin(), v.end(), 17);  
if (it != v.end()) {  
    cout << "Element found at index: " << (it - v.begin());  
} else {  
    cout << "Element not found";  
}
```

Output

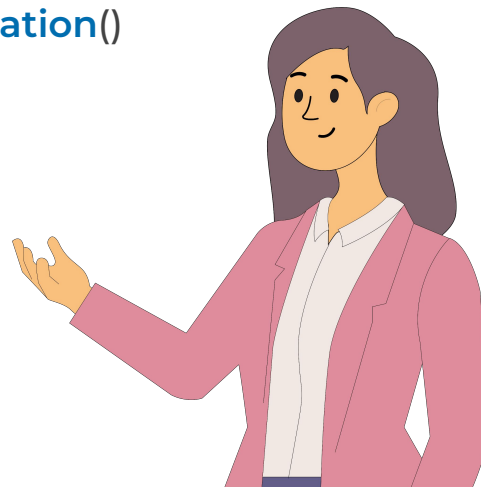
Element found at index: 2

Permutations (next, previous, find all)



Permutations in C++ STL

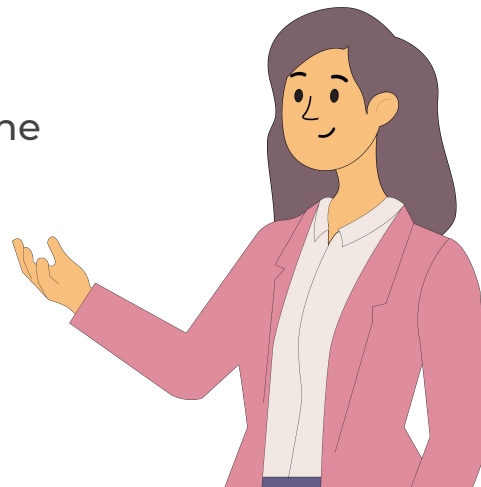
- ✓ A permutation is a **rearrangement** of elements in a given order.
- ✓ C++ provides the **next_permutation()** and **prev_permutation()** functions to generate lexicographic permutations.



About

What does `next_permutation()` do?

- ✓ The `next_permutation()` function is part of the `<algorithm>` header in **C++**.
- ✓ It rearranges elements in the given range to generate the **next lexicographical permutation** of the sequence.



Generating the Next Permutation

About

Input



- A range defined by two iterators: (**first**, **last**)
- **first**: Iterator pointing to the beginning of the range.
- **last**: Iterator pointing one past the last element of the range.

Output



- **Returns true** if the previous permutation exists and modifies the range in-place.
- **Returns false** if the given range is the smallest permutation (**sorted in ascending order**), in which case the sequence is rearranged to the largest permutation (**sorted in descending order**).

Important note



The function modifies the original sequence directly.

If no next permutation exists, the sequence is rearranged in descending order.

Generating the Next Permutation

Example

C++

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    string str = "abc";
    if (next_permutation(str.begin(), str.end())) {
        cout << "Next permutation: " << str << endl;
    } else {
        cout << "No next permutation exists" << endl;
    }
    return 0;
}
```

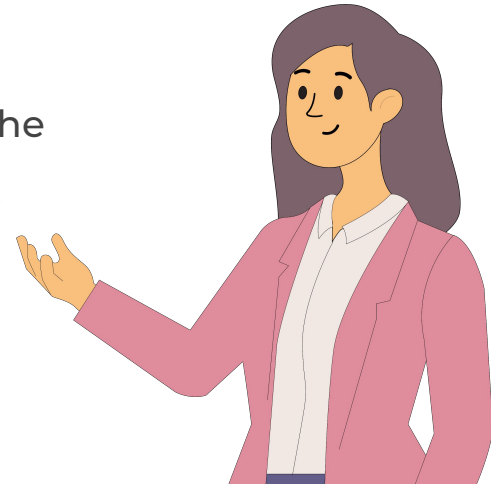
Output

Next permutation: acb

About

What does `prev_permutation()` do?

- ✓ The `prev_permutation()` function is part of the `<algorithm>` header in **C++**.
- ✓ It rearranges elements in the given range to generate the **previous lexicographical permutation** of the sequence.



Generating the Previous Permutation

About

Input



- A range defined by two iterators: (**first**, **last**)
- **first**: Iterator pointing to the beginning of the range.
- **last**: Iterator pointing one past the last element of the range.

Output



- **Returns true** if the previous permutation exists and modifies the range in-place.
- **Returns false** if the given range is the smallest permutation (**sorted in ascending order**), in which case the sequence is rearranged to the largest permutation (**sorted in descending order**).

Important note



The function modifies the original sequence directly.

If no previous permutation exists, the sequence is rearranged in descending order.

Generating the Previous Permutation

Example

C++

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    string str = "cba";
    if (prev_permutation(str.begin(), str.end())) {
        cout << "Previous permutation: " << str << endl;
    } else {
        cout << "No previous permutation exists" << endl;
    }
    return 0;
}
```

Output

Previous permutation: cab

Finding All Permutations

Example

C++

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    string str = "bac";
    sort(str.begin(), str.end());
    do {
        cout << str << endl;
    } while (next_permutation(str.begin(), str.end()));
    return 0;
}
```

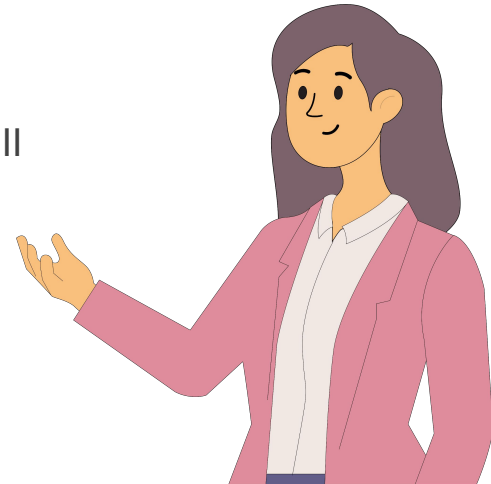
Output

abc
acb
bac
bca
cab
cba

About

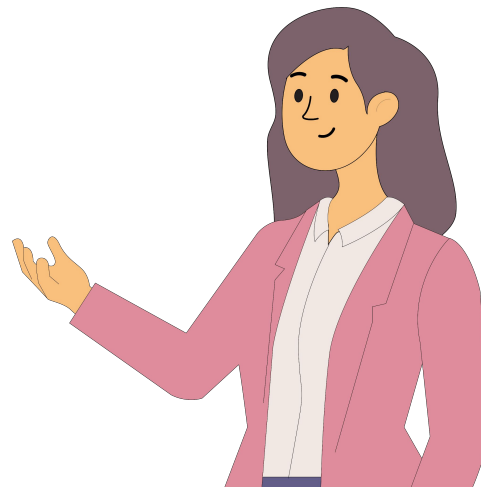
What does **Sorting Required?**

- ✓ The sorting step ensures that the permutations are generated in **lexicographical order** when using **next_permutation()**.
- ✓ If the input **string** is not in sorted order, the function will continue from its **current arrangement**, missing some permutations at the star.



Supported Data Structures

- ✓ **Applicable to Various Containers:** `next_permutation` works with `strings`, `arrays`, and `vectors`, allowing permutation generation for different data types.
- ✓ **Supports Bidirectional Iterators:** It can be used with any container that provides `bidirectional iterators`, including `std::list`.
- ✓ **Flexible Across STL Containers:** Compatible with standard containers like `std::deque`, `std::string`, and `std::vector`, making it versatile for sequence manipulation.



Finding All Permutations for an Array

Example

C++

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    int arr[] = {1, 2, 3};
    int n = 3;
    sort(arr, arr + n);
    do {
        for (int i = 0; i < n; i++)
            cout << arr[i] << " ";
        cout << endl;
    } while (next_permutation(arr, arr + n));
    return 0;
}
```

Output

1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1

Key Takeaways

- ✓ *Sorting in C++ STL:* Using `sort(start, end)` from `<algorithm>` to efficiently sort arrays and vectors in ascending or descending order.
- ✓ *Custom Sorting with Comparators:* Implementing a **boolean comparator function** to define custom sorting logic, such as sorting pairs or **2D vectors**.
- ✓ *Finding Maximum and Minimum Elements:* Using `max_element()` and `min_element()` to find the **largest** and **smallest** values in an **array** or **vector**.



Key Takeaways

- ✓ *Searching for Elements:* Using `find()` to search for a target value in a container and `return` its position.
- ✓ *Generating Permutations:* Using `next_permutation()` and `prev_permutation()` to generate **lexicographic permutations** of sequences.
- ✓ *Working with 2D Vectors:* Sorting **entire rows, specific columns**, or using custom sorting conditions for **multi-dimensional data**.





THANK YOU



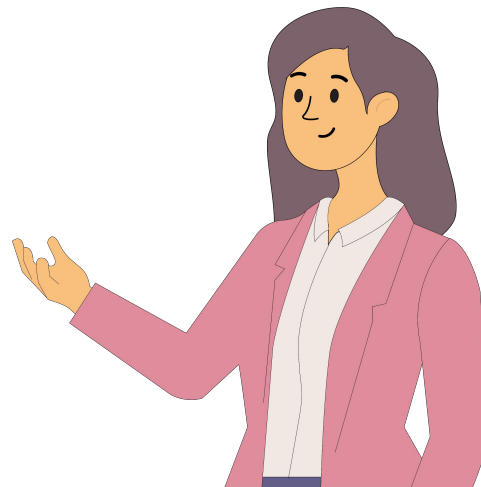


ALL THE BEST



Problem Statement

- ✓ Given a vector of **unique** integers '**arr**' of length '**n**', you need to **sort** the **vector** in **non-decreasing order** based on the **absolute difference** from '**0**'.
- ✓ If **two numbers** have the **same absolute difference**, the **smaller number** should come **first**.



Custom Sorting of Vector

Example

Input



$n = 8$

$arr = [2, 6, -2, 5, -5, 7, 11, -22]$

Output



$[-2, 2, -5, 5, 6, 7, 11, -22]$

Calculate absolute difference values with '0':

2	→	2	=	2
6	→	6	=	6
-2	→	-2	=	2
5	→	5	=	5
-5	→	-5	=	5
7	→	7	=	7
11	→	11	=	11
-22	→	-22	=	22

Sort by absolute values: $[-2, 2, -5, 5, 6, 7, 11, -22]$

- -2 before 2 (same absolute value, smaller first).
- -5 before 5 (same absolute value, smaller first).

Example

C++



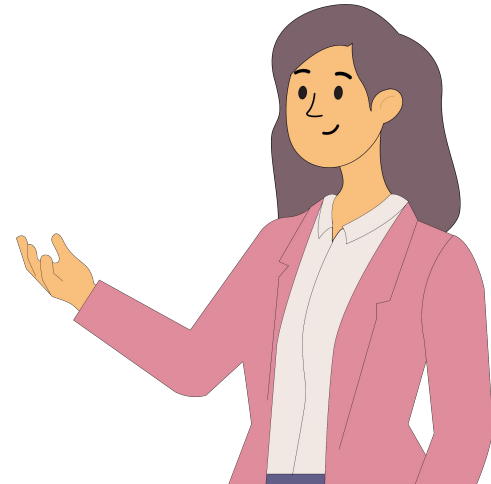
```
#include <bits/stdc++.h>
using namespace std;

bool customSort(int a, int b) {
    if (abs(a) == abs(b))
        return a < b;
    return abs(a) < abs(b);
}

void customSorting(vector<int>& arr) {
    sort(arr.begin(), arr.end(), customSort);
}
```

Problem Statement

- ✓ Given a vector nums of size n containing integers, check if the difference between the **maximum element** and the **first element** exists in the vector. If found, print "Present".



Problem Statement

Input



$n = 5$

$\text{nums} = [3, 1, 7, 4, 2]$

Output



Present

*The first element is **3**, the maximum is **7**, and since $7 - 3 = 4$ exists in the **vector**, the output is "**Present**".*

Max Element Solution

C++ 

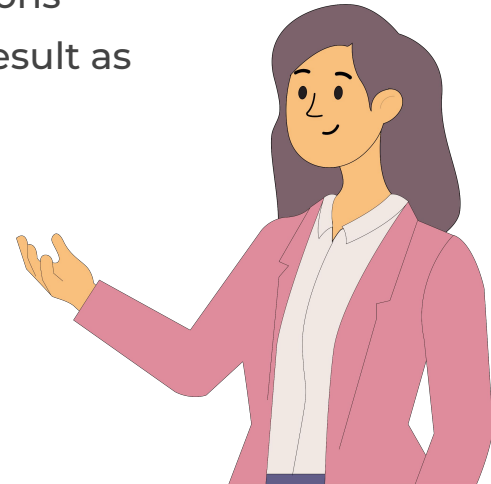
```
#include<bits/stdc++.h>
using namespace std;

void checkPresence(vector<int>& nums) {
    int first = nums[0];
    int maxElement = *max_element(nums.begin(), nums.end());
    int difference = maxElement - first;

    if (find(nums.begin(), nums.end(), difference) != nums.end()) {
        cout << "Present" << endl;
    } else {
        cout << "Not Present" << endl;
    }
}
```

Problem Statement

- ✓ Given a **string s**, find the next **k lexicographically** greater permutations of **s**. If there are fewer than **k** permutations available, return only the ones that exist. **Return** the result as a vector of strings in sorted order.



Problem Statement

Input



`s = "abc"`

`k = 2`

Output



`["acb", "bac"]`

The next permutation after "**abc**" is "**acb**", followed by "**bac**".

Finding All Permutations for an Array

Solution

C++ 

```
#include<bits/stdc++.h>
using namespace std;

vector<string> findNextPermutations(string s, int k) {
    vector<string> result;

    while (k-- && next_permutation(s.begin(), s.end())) {
        result.push_back(s);
    }

    return result;
}
```